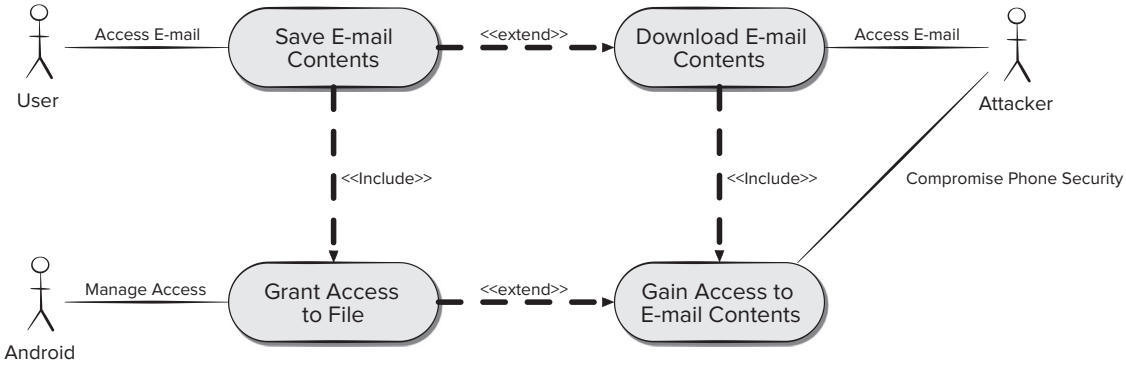
**FIGURE 18.3** **Misuse case (exploited by DroidCleaner): Data in an e-mail stored on the smartphone is stolen**




a software team to implement all defined mitigation strategies at once. In such cases, the strategies can be prioritized and implemented incrementally. The team can also exclude certain cases as being extremely unlikely.

Templates for misuse and abuse cases appear in a number of references. They can be text or diagrams and may be supported by tools. Good sources for templates are in materials by Sindre and Opdahl [Sin01] and Alexander [Ale02].

## 18.6 SECURITY RISK ANALYSIS

A wide variety of security risk assessment methods have been proposed. Typical examples include SEI CERT's Security Engineering Risk Analysis (SERA) method[3] and the NIST Risk Management Framework (RMF).[4]

RMF has emerged as an approach that is widely used, providing guidelines for the users. The RMF steps for security are:

- **Categorize** the information system and the information processed, stored, and transmitted by that system based on an impact analysis.
- **Select** an initial set of baseline security controls for the information system based on the security categorization; using an organizational assessment of risk and local conditions, tailor and supplement the security control baseline as needed.
- **Implement** the security controls, and describe how the controls are employed within the information system and its operational environment.
- **Assess** the security controls using appropriate assessment procedures to determine the extent to which the controls are implemented correctly, operating as intended, and producing the desired outcome with respect to meeting the security requirements for the system.

3 See https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=485410.
4 See https://csrc.nist.gov/publications/detail/sp/800-37/rev-1/final.

- **Authorize** the information system operation based on a determination of the risk to organizational operations and assets, individuals, or other organizations (including national defense), from the operation of the information system that this risk is acceptable.
- **Monitor** the security controls in the information system on an ongoing basis including assessing control effectiveness, documenting changes to the system or its environment of operation, conducting security impact analyses of the associated changes, and reporting the security state of the system to designated organizational officials.

It's important to note that NIST also provides a set of security controls to select from, thus simplifying the risk assessment work. Recently, the RMF has been modified to include privacy concerns.

## 18.7 Threat Modeling, Prioritization, and Mitigation

A threat modeling method (TMM) is an approach for creating an abstraction of a software system, aimed at identifying attackers' abilities and goals, and using that abstraction to generate and catalog possible threats that the system must mitigate [Shu16].

STRIDE (an acronym for six threat categories) is representative of a number of threat modeling methods [Mea18] and is the most well-established TMM, representing the state of the practice. At its core, STRIDE requires breaking down a system into its various elements, assessing each of these elements for their vulnerability to threats, and then mitigating these threats [Her06]. In practice, a typical STRIDE implementation includes modeling a system with data flow diagrams (DFDs),[5] mapping the DFD elements to the six threat categories, determining the specific threats via checklists or threat trees, and documenting the threats and steps for their prevention [Sca15]. STRIDE can be implemented manually; however, a free Microsoft Secure Development Lifecycle (SDL) Threat Modeling Tool [Mic17] can also be used. Table 18.2 identifies the security property associated with each of the six threat categories.

**TABLE 18.2**

**Threat categories and security properties**

| Threat | Security Property |
|---|---|
| Spoofing | Authentication |
| Tampering | Integrity |
| Repudiation | Nonrepudiation |
| Information disclosure | Confidentiality |
| Denial of service | Availability |
| Elevation of privilege | Authorization |

5 A brief tutorial on data flow diagrams can be downloaded from https://ratandon.mysite.syr.edu/cis453/notes/DFD_over_Flowcharts.pdf.

**TABLE 18.3**
**Threat categories of DFD system elements**

| Element | Spoofing | Tampering | Repudiation | Information Disclosure | Denial of Service | Elevation of Privilege |
|---|---|---|---|---|---|---|
| Data flows | | X | | X | X | |
| Data stores | | X | | X | X | |
| Processes | X | X | X | X | X | X |
| External entity | X | | X | | | |

DFDs are designed to show how a system works by using standard symbols to graphically represent the interaction between data stores (e.g., databases, files, registries), processes (e.g., DLLs, Web services), data flows (e.g., function calls, remote procedure calls), and external entities (e.g., people, other systems) [Sho14]. Once complete, each of these system elements in turn can be associated with one or more relevant threat categories, as depicted in Table 18.3.

In the next stage, the typical STRIDE user works through a checklist (that may be in the form of a threat tree) of specific threats that are associated with each match between a DFD element and threat category. Such checklists are accessible through STRIDE reference books or tools.

Once the threats have been identified, mitigation strategies can be developed and prioritized. Typically, prioritization is based on cost and value considerations. Considering the cost of implementing the mitigation strategy is important, but it's equally important to also consider the cost of *not* implementing it, which is reflected in value. Remember that risks that are realized result in costs that are not only expressed in terms of dollars, but could also reflect loss of reputation, loss of trust, and even loss of life.

## 18.8 ATTACK SURFACE

An *attack surface* can be defined[6] in the following manner:

> The attack surface describes all of the different points where an attacker could get into a system, and where they could get data out.
>
> The attack surface of an application is:
>
> 1. the sum of all paths for data/commands into and out of the application, and
> 2. the code that protects these paths (including resource connection and authentication, authorization, activity logging, data validation and encoding), and
> 3. all valuable data used in the application, including secrets and keys, intellectual property, critical business data, personal data and PII, and
> 4. the code that protects these data (including encryption and checksums, access auditing, and data integrity and operational security controls). [OWA18]

6 See https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.md.

The OWASP Foundation [OWA18] states that attack surface analysis is:

> . . . targeted to be used by developers to understand and manage application security risks as they design and change an application, as well as by application security specialists doing a security risk assessment. The focus here is on protecting an application from external attack—it does not take into account attacks on the users or operators of the system (e.g., malware injection, social engineering attacks), and there is less focus on insider threats, although the principles remain the same. The internal attack surface is likely to be different to the external attack surface and some users may have a lot of access.
>
> Attack Surface Analysis is about mapping out what parts of a system need to be reviewed and tested for security vulnerabilities. The point of attack surface analysis is to understand the risk areas in an application, to make developers and security specialists aware of what parts of the application are open to attack, to find ways of minimizing this, and to notice when and how the Attack Surface changes and what this means from a risk perspective.

## 18.9 SECURE CODING

*Secure coding* is just what the name implies—coding in such a way that vulnerabilities are not introduced as a result of coding errors. It's not surprising that most software vulnerabilities occur because of sloppy and erroneous coding practices, many of which can be easily avoided.

For example, a condition known as *buffer overflow* results from one of the most well-known and common coding errors. OWASP[7] describes it as follows:

> A buffer overflow condition exists when a program attempts to put more data in a buffer than it can hold or when a program attempts to put data in a memory area past a buffer. In this case, a buffer is a sequential section of memory allocated to contain anything from a character string to an array of integers. Writing outside the bounds of a block of allocated memory can corrupt data, crash the program, or cause the execution of malicious code.

Buffer overflow is just one example of coding errors that can result in vulnerabilities. Fortunately, a number of coding standards now exist to provide guidance on secure coding. The SEI/CERT website[8] provides a list of the top-10 secure coding practices:

1. **Validate input.** Validate input from all untrusted data sources.
2. **Heed compiler warnings.** Compile code using the highest warning level available for your compiler and eliminate warnings by modifying the code.
3. **Architect and design for security policies.** Create a software architecture and design your software to implement and enforce security policies.
4. **Keep it simple.** Keep the design as simple and as small as possible.
5. **Default deny.** Base access decisions on permission rather than exclusion.

7 See https://www.owasp.org/index.php/Buffer_overflow_attack.

8 See https://wiki.sei.cmu.edu/confluence/display/seccode/Top+10+Secure+Coding+Practices.

6. **Adhere to the principle of least privilege.** Every process should execute with the least set of privileges necessary to complete the job.
7. **Sanitize data sent to other systems.** Sanitize all data passed to complex subsystems such as command shells, relational databases, and commercial off-the-shelf (COTS) components.
8. **Practice defense in depth.** Manage risk with multiple defensive strategies.
9. **Use effective quality assurance techniques.**
10. **Adopt a secure coding standard.**

SEI CERT and others also provide secure coding standards.[9] In addition to using a secure coding standard, you should inspect for coding errors that lead to vulnerabilities. This is just a natural add-on to your normal code inspection and review process (Chapter 16). *Static analysis tools*[10] are used to automatically analyze code and are another mechanism for detecting vulnerabilities due to coding errors.

## 18.10 MEASUREMENT

Developing adequate measures of software security is a difficult problem, and one for which there are differing viewpoints. On the one hand, you can look at the development processes followed and assess whether the resultant software is likely to be secure. On the other hand, you can look at the incidence of vulnerabilities and successful break-ins and measure those as a way of assessing software security. However, neither of these measurement approaches will allow you to say with 100 percent certainty that our software is secure. When you add supporting software such as operating systems and external interoperable systems, the measurement of software security becomes even more difficult. Nevertheless some progress has been made.

Measures of software quality can go a long way toward measuring software security. Specifically, vulnerabilities invariably point to software defects. Although not all software defects are security problems, vulnerabilities in software generally result from a defect of some kind, whether it is in the requirements, architecture, or code. Therefore, measures such as defect and vulnerability counts [Woo14] are useful. Microsoft uses measures such as attack surface analysis and tries to keep the attack surface (places where software can be compromised) to a minimum.

Just as use of maturity models such as CMMI (Chapter 28) suggests that higher-quality software will result, mature security development processes, such as those emphasized by BSIMM,[11] will result in more secure software. In some cases, organizations are encouraged to identify the unique set of security metrics that are relevant to them. BSIMM makes reference to this, as does SAMM.[12]

9 See https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards.

10 A list of commercially available tools can be found at https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis.

11 See https://www.bsimm.com/.

12 See https://www.owasp.org/index.php/OWASP_SAMM_Project.

It's important to note that none of the measurement characteristics implied by various maturity models is perfect. If you follow good secure software development processes, does that guarantee that the software is secure? No! If you find a lot of vulnerabilities, does that mean that most of them have been found or that there are still more to be found because this is a particularly poor piece of software? There are no simple answers. However, to assess vulnerabilities and associated software security, we have to collect data so that patterns can be analyzed over time. If we don't collect data about software security, we will never be able to measure its improvement.

Tables 18.4 and 18.5 provide examples of how to assess software security during each life-cycle phase. The full tables and discussion can be found in [Mea17] and [Alb10].

**TABLE 18.4**

**Examples of life-cycle-phase measures**

| Life-Cycle Phase | Example Software Security Measures |
|---|---|
| Requirements engineering | Percentage of relevant software security principles reflected in requirements-specific actions (assuming security principles essential for a given development project have been selected) |
| | Percentage of security requirements that have been subject to analysis (risk, feasibility, cost-benefit, performance trade-offs) prior to being included in the specification |
| | Percentage of security requirements covered by attack patterns, misuse and abuse cases, and other specified means of threat modeling and analysis |
| Architecture and design | Percentage of architectural and design components subject to attack surface analysis and measurement |
| | Percentage of architectural and design components subject to architectural risk analysis |
| | Percentage of high-value security controls covered by a security design pattern |

**TABLE 18.5**

**Example measures based on the seven principles of evidence**

| Principle | Description |
|---|---|
| Risk | Number of active and latent threats, categorized |
| | Incidents reported by category of threat |
| | Likelihood of occurrence for each threat category |
| | Financial and/or human safety estimate of impact for each threat category |
| Trusted dependencies | Number of levels of subcontracting in the supply chain (in other words, have the subcontractors, in turn, executed subcontracts, and what is the depth of this activity?) |
| | Number of suppliers by level |
| | Hierarchical and peer dependencies between suppliers by level |
| | Number of (vetted) trusted suppliers in the supply chain by level |